

COD 23-25 LightGBM

Elliot Dean
Office of the Chief Financial Officer | Office of the Assessor
City of Detroit

24 February 2026

Contents

Load Necessary Libraries	1
Load Data	2
Model I	6
Model II	8
Model III	9
Model IV	10
Model V	11
Model VI	12

Load Necessary Libraries

```
library(tidyverse)    # data wrangling & ggplot2
library(magrittr)
library(janitor)
library(kableExtra)   # Create unique tables
library(caret)         # train/test split, cross-validation
library(broom)        # tidy model output
library(car)           # VIF, Durbin-Watson
library(lmtest)       # Breusch-Pagan test
library(tidymodels)   # collection of packages for modeling and machine learning
library(xgboost)      # Extreme Gradient Boosting
library(bundle)       # Serialize Model Objects with a Consistent Interface
library(data.table)   # fast I/O + data handling
library(lightgbm)     # LightGBM R package
library(pROC)         # AUC
library(here)         # Find files within a project
library(Matrix)       # sparse matrices play nicely with one-hot encoding
library(diffdf)       # tools for working with data frame difference

# ---- 1. Load data & normalize names ----
DATA_PATH <- read_csv(here("data/city", "salesSFapr2023toMarch2025Updated.csv")) |>
  rename_with(tolower)    # clean_names()

OUT_DIR <- here("models/codmods")
```

Load Data

```
load(file = here("data/rdata", "CODsalesSFapr2023Mar2025.RData"))
```

```
OUT_DIR <- here("models/codmods")
```

```
set.seed(42)
```

```
# -----  
# Core: build design matrix from formula  
# -----  
# This is the key AVM-ish piece:  
# - formula defines predictors  
# - model.matrix makes your X  
# - terms + xlevels are saved for consistent future encodes  
#  
make_design <- function(df,  
                        formula,  
                        terms_obj = NULL,  
                        xlevels = NULL,  
                        handle_new_levels = c("na", "drop", "error")) {  
  
  handle_new_levels <- match.arg(handle_new_levels)  
  
  # lock terms (training or reuse at inference)  
  if (is.null(terms_obj)) {  
    terms_obj <- terms(formula, data = df)  
  }  
  
  # --- CRITICAL: pre-sanitize factor columns BEFORE model.frame(xlev=...) ---  
  # If we're using saved xlevels (test/inference), make sure df's factor columns  
  # cannot contain unseen levels, otherwise model.frame will error immediately.  
  if (!is.null(xlevels)) {  
    fac_vars <- intersect(names(xlevels), names(df))  
  
    for (v in fac_vars) {  
      # Work on a temporary vector so we don't mutate df unexpectedly unless needed  
      col <- df[[v]]  
  
      # Ensure it's character so we can safely compare levels/values  
      if (is.factor(col)) col_chr <- as.character(col) else col_chr <- col  
  
      # Only process if it's character/factor-like (ignore numeric-coded cats)  
      if (is.character(col_chr)) {  
        unseen <- !is.na(col_chr) & !(col_chr %in% xlevels[[v]])  
  
        if (any(unseen)) {  
          new_vals <- unique(col_chr[unseen])  
  
          if (handle_new_levels == "error") {  
            stop(sprintf(  
              "New factor levels found in '%s': %s",  
              v, paste(new_vals, collapse = ", ")  
            ))  
          }  
  
          if (handle_new_levels == "drop") {  
            # drop offending rows  
          }  
        }  
      }  
    }  
  }  
}
```

```

    df <- df[!unseen, , drop = FALSE]
    # need to update col_chr after dropping
    col_chr <- df[[v]]
    if (is.factor(col_chr)) col_chr <- as.character(col_chr)
  } else if (handle_new_levels == "na") {
    # convert unseen to NA (LightGBM handles missing well)
    col_chr[unseen] <- NA_character_
  }
}

# Now lock factor levels to training levels
df[[v]] <- factor(col_chr, levels = xlevels[[v]])
}
}

# Build model.frame using locked terms (now safe with xlev)
mf <- model.frame(terms_obj, data = df, na.action = na.pass, xlev = xlevels)

# Capture xlevels during training
if (is.null(xlevels)) {
  xlevels <- .getXlevels(terms_obj, mf)
}

# Response (if present)
y <- model.response(mf)

# Sparse one-hot design matrix
X <- Matrix::sparse.model.matrix(terms_obj, mf)

# Drop intercept if present
if ("(Intercept)" %in% colnames(X)) {
  X <- X[, setdiff(colnames(X), "(Intercept)"), drop = FALSE]
}

list(
  X = X,
  y = y,
  terms = terms_obj,
  xlevels = xlevels,
  feature_names = colnames(X)
)
}

# --- helper: set LightGBM feature names compatibly ---
set_lgb_feature_names <- function(dset, feature_names) {
  # Many builds support setting fields
  if (exists("lgb.Dataset.set.field", where = asNamespace("lightgbm"), inherits = FALSE)) {
    lightgbm::lgb.Dataset.set.field(dset, field_name = "feature_name", data = feature_names)
  }
  dset
}

# -----
# Metrics (regression)
# -----
rmse <- function(y, yhat) sqrt(mean((y - yhat)^2, na.rm = TRUE))
mae <- function(y, yhat) mean(abs(y - yhat), na.rm = TRUE)

```

```

r2 <- function(y, yhat) {
  ss_res <- sum((y - yhat)^2, na.rm = TRUE)
  ss_tot <- sum((y - mean(y, na.rm = TRUE))^2, na.rm = TRUE)
  1 - ss_res/ss_tot
}

# -----
# Train function (AVM regression)
# -----
train_lgbm_avm <- function(df, formula, test_frac = 0.2, handle_new_levels = "na",
  params = list(
    objective = "regression", metric = "rmse", learning_rate = 0.05,
    num_leaves = 64, feature_fraction = 0.85, bagging_fraction = 0.85,
    bagging_freq = 1, min_data_in_leaf = 50, verbosity = -1
  ), num_boost_round = 5000, early_stopping_rounds = 100) {

  # Basic NA strategy: # keep NAs in predictors; model.matrix will keep NA rows,
  # and LightGBM can handle missing values. # But response must not be NA.
  resp_name <- all.vars(formula)[1]
  df <- df[!is.na(df[[resp_name]]), ]

  # Split
  n <- nrow(df) # or df[, .N]
  idx_test <- sample.int(n, size = floor(test_frac * n))
  idx_train <- setdiff(seq_len(n), idx_test)
  train_df <- df[idx_train, ]
  test_df <- df[idx_test, ]

  # Build train design (captures terms + xlevels)
  tr <- make_design(train_df, formula, handle_new_levels = handle_new_levels)

  # Build test design using locked terms + xlevels
  te <- make_design(test_df, formula, terms_obj = tr$terms,
    xlevels = tr$xlevels, handle_new_levels = handle_new_levels)

  # Ensure feature alignment (model.matrix guarantees order if terms identical,
  # but we assert to be safe)
  if (!identical(tr$feature_names, te$feature_names)) {
    # Align test columns to train columns (in case of dropped rows or subtle diffs)
    # Add missing cols in test, drop extra cols.
    missing <- setdiff(tr$feature_names, te$feature_names)
    extra <- setdiff(te$feature_names, tr$feature_names)

    if (length(missing) > 0) {
      add <- Matrix::Matrix(0, nrow = nrow(te$X), ncol = length(missing), sparse = TRUE)
      colnames(add) <- missing
      te$X <- cbind(te$X, add)
    }
    if (length(extra) > 0) {
      te$X <- te$X[, setdiff(colnames(te$X), extra), drop = FALSE]
    }
    # Reorder to match train
    te$X <- te$X[, tr$feature_names, drop = FALSE]
    te$feature_names <- colnames(te$X)
  }

  dtrain <- lgb.Dataset(data = tr$X, label = tr$y)
  dvalid <- lgb.Dataset(data = te$X, label = te$y)

```

```

# ensure column names exist (important for importance tables)
colnames(tr$X) <- tr$feature_names
colnames(te$X) <- tr$feature_names

# set names on datasets (safe no-op if not supported)
dtrain <- set_lgb_feature_names(dtrain, tr$feature_names)
dvalid <- set_lgb_feature_names(dvalid, tr$feature_names)

booster <- lgb.train(
  params = params, data = dtrain, nrounds = num_boost_round, valids = list(valid = dvalid),
  early_stopping_rounds = early_stopping_rounds, eval_freq = 50
)

pred_test <- predict(booster, te$X)

metrics <- list(
  rmse = rmse(te$y, pred_test), mae = mae(te$y, pred_test), r2 = r2(te$y, pred_test)
)

importance <- lgb.importance(booster, percentage = TRUE)

list(
  booster = booster, formula = formula, terms = tr$terms, xlevels = tr$xlevels,
  feature_names = tr$feature_names, metrics = metrics, importance = importance
)
}

```

```

# -----
# Save bundle (model + encoding contract)
# -----
save_avm_bundle <- function(fit, prefix = "lgbm_avm") {
  ts <- stamp()
  model_file <- file.path(OUT_DIR, sprintf("%s_%s.txt", prefix, ts))
  bundle_file <- file.path(OUT_DIR, sprintf("%s_%s_bundle.rds", prefix, ts))
  imp_file <- file.path(OUT_DIR, sprintf("%s_%s_importance.csv", prefix, ts))

  lgb.save(fit$booster, model_file)

  bundle <- list(
    model_file = model_file,
    formula = fit$formula,
    terms = fit$terms,
    xlevels = fit$xlevels,
    feature_names = fit$feature_names,
    best_iter = fit$booster$best_iter
  )
  saveRDS(bundle, bundle_file)

  fwrite(fit$importance, imp_file)

  list(model_file = model_file, bundle_file = bundle_file, imp_file = imp_file)
}

```

```

# -----
# Predict helper (safe inference)
# -----
# Provide df_new (one row or many); uses saved terms/xlevels to build X in same feature space
predict_avm <- function(bundle_rds, df_new, handle_new_levels = "na") {

```

```

bundle <- readRDS(bundle_rds)
booster <- lgb.load(bundle$model_file)

des <- make_design(df_new, bundle$formula,
                  terms_obj = bundle$terms,
                  xlevels = bundle$xlevels,
                  handle_new_levels = handle_new_levels)

# Align columns to training feature set
X <- des$X
if (!identical(colnames(X), bundle$feature_names)) {
  missing <- setdiff(bundle$feature_names, colnames(X))
  extra <- setdiff(colnames(X), bundle$feature_names)

  if (length(missing) > 0) {
    add <- Matrix::Matrix(0, nrow = nrow(X), ncol = length(missing), sparse = TRUE)
    colnames(add) <- missing
    X <- cbind(X, add)
  }
  if (length(extra) > 0) {
    X <- X[, setdiff(colnames(X), extra), drop = FALSE]
  }
  X <- X[, bundle$feature_names, drop = FALSE]
}

predict(booster, X)
}

```

```

# -----
# Optional: AVM-ish "newdata_from_sale" helper
# -----
# Given a full df, pull one record by an id and return the row as newdata for predict_avm()
newdata_from_id <- function(df, id_col, id_value) {
  stopifnot(id_col %in% names(df))
  row <- df[get(id_col) == id_value]
  if (nrow(row) == 0) stop("No rows found for that id.")
  row
}

```

Model I

```

# =====
# MAIN EXAMPLE (edit this to match your AVM variables)
# =====
# message("Reading data: ", DATA_PATH)
# df <- fread(DATA_PATH)
df <- dfa

# Example AVM formula: # - choose EXACTLY the variables you want
# - you can include factor() for categorical predictors
# - you can do transforms inside formula (log, poly, interactions, etc.)
#
# Typical price AVM:
# ln_price ~ living_area + lot_area + beds + baths + year_built + factor(ecf) + factor(zip) + dist_downtown

```

```
#
# If you train on price directly: #   sale_price ~ ...
#
formula_time <- ln_sppsf ~ arterial + single_family1story + half_duplex1story + half_duplex2story +
  smallest + small + large + largest + two_plus_baths + powder_room +
  good + fair + poor + very_poor_unsound + qabove_avg + qbelow_avg +
  sresb_yearbuilt + crawl + slab + ln_garagespaces +
  fireplace + hot_water + elec_baseboard + wall + central_air + months

message("Training LightGBM AVM with formula-driven features...")
mod_time <- train_lgbm_avm(df, formula = formula_time)

message("\nHoldout metrics:")
print(mod_time$metrics)
```

```
## $rmse
## [1] 0.5202728
##
## $mae
## [1] 0.4167283
##
## $r2
## [1] 0.1620253
```

```
message("\nTop 20 features by Gain:")
print(head(mod_time$importance[order(-Gain)], 20))
```

	Feature	Gain	Cover	Frequency
	<char>	<num>	<num>	<num>
## 1:	sresb_yearbuilt	0.28060563	0.27581146	0.295948204
## 2:	months	0.17273335	0.14055941	0.217836257
## 3:	fair	0.09612367	0.06825184	0.030075188
## 4:	qabove_avg	0.06521098	0.04683670	0.011904762
## 5:	ln_garagespaces	0.06007884	0.05345593	0.084795322
## 6:	poor	0.06002798	0.05157658	0.018170426
## 7:	fireplace	0.03776341	0.04714480	0.041979950
## 8:	good	0.02521385	0.03939074	0.011278195
## 9:	large	0.02391370	0.01761699	0.040935673
## 10:	single_family1story	0.02373334	0.01629891	0.037593985
## 11:	central_air	0.02272167	0.03367258	0.024227235
## 12:	qbelow_avg	0.02061610	0.01993884	0.039682540
## 13:	largest	0.01579773	0.01466788	0.016499582
## 14:	powder_room	0.01395899	0.01434902	0.021929825
## 15:	two_plus_baths	0.01357072	0.01172969	0.006683375
## 16:	smallest	0.01271186	0.02592064	0.021929825
## 17:	crawl	0.01260133	0.02324868	0.008980785
## 18:	small	0.01228125	0.01850987	0.030075188
## 19:	hot_water	0.01150527	0.01015525	0.016708438
## 20:	very_poor_unsound	0.01016212	0.03273980	0.005430242
##	Feature	Gain	Cover	Frequency

```
# art <- save_avm_bundle(fit)
# message("\nSaved artifacts:")
# print(art)
```

Model II

```
formula_ln_price_no_ecf <- ln_price ~
  ln_landratio + ln_base_bldgsize_ratio +
  arterial + single_family1story + half_duplex1story + half_duplex2story +
  smallest + small + large + largest + two_plus_baths + powder_room +
  good + fair + poor + very_poor_unsound + qabove_avg + qbelow_avg +
  sresb_yearbuilt + crawl + slab + ln_garagespaces +
  fireplace + hot_water + elec_baseboard + wall + central_air + months

message("Training LightGBM AVM with formula-driven features...")
mod_ln_price_no_ecf <- train_lgbm_avm(df, formula = formula_ln_price_no_ecf)

message("\nHoldout metrics:")
print(mod_ln_price_no_ecf$metrics)
```

```
## $rmse
## [1] 0.5112981
##
## $mae
## [1] 0.4124882
##
## $r2
## [1] 0.3654942
```

```
message("\nTop 20 features by Gain:")
print(head(mod_ln_price_no_ecf$importance[order(-Gain)], 20))
```

##	Feature	Gain	Cover	Frequency
##	<char>	<num>	<num>	<num>
## 1:	ln_base_bldgsize_ratio	0.370721889	0.223366498	0.236631769
## 2:	sresb_yearbuilt	0.121302697	0.177608226	0.183609141
## 3:	ln_landratio	0.116494737	0.211102715	0.196780367
## 4:	months	0.090287847	0.086339100	0.159405606
## 5:	largest	0.047762165	0.006990019	0.002138917
## 6:	fair	0.043789953	0.036426395	0.028706518
## 7:	fireplace	0.039152593	0.027961701	0.020601148
## 8:	qabove_avg	0.036832755	0.011046209	0.005178431
## 9:	ln_garagespaces	0.022810675	0.026215508	0.047619048
## 10:	poor	0.017441259	0.031116704	0.011707756
## 11:	large	0.012467067	0.005571671	0.007992795
## 12:	central_air	0.012464517	0.019164799	0.016323314
## 13:	two_plus_baths	0.011852951	0.003201870	0.002364066
## 14:	good	0.010848374	0.021318008	0.006416751
## 15:	qbelow_avg	0.006825383	0.011702935	0.014296972
## 16:	smallest	0.006493301	0.003052769	0.002138917
## 17:	single_family1story	0.006477969	0.008563676	0.015422718
## 18:	powder_room	0.006164676	0.003984042	0.004165259
## 19:	hot_water	0.005349785	0.011970533	0.011595182
## 20:	small	0.004529368	0.003448549	0.006529326
##	Feature	Gain	Cover	Frequency

```
# art <- save_avm_bundle(fit)
# message("\nSaved artifacts:")
# print(art)
```

Model III

```
formula_base <- ln_price ~
  ln_landratio + ln_base_bldgsize_ratio + arterial +
  single_family1story + half_duplex1story + half_duplex2story + smallest + small + large + largest +
  two_plus_baths + powder_room + good + fair + poor + very_poor_unsound +
  qabove_avg + qbelow_avg + sresb_yearbuilt + crawl + slab + ln_garagespaces +
  fireplace + hot_water + elec_baseboard + wall + central_air + months + ecf

message("Training LightGBM AVM with formula-driven features...")
mod_base <- train_lgbm_avm(df, formula = formula_base)

message("\nHoldout metrics:")
print(mod_base$metrics)
```

```
## $rmse
## [1] 0.4953771
##
## $mae
## [1] 0.3980906
##
## $r2
## [1] 0.43253
```

```
message("\nTop 20 features by Gain:")
print(head(mod_base$importance[order(-Gain)], 20))
```

##	Feature	Gain	Cover	Frequency
##	<char>	<num>	<num>	<num>
## 1:	ln_base_bldgsize_ratio	0.283124649	0.108599103	0.173004454
## 2:	sresb_yearbuilt	0.101015830	0.101170352	0.158844353
## 3:	ln_landratio	0.096677953	0.106272492	0.155761105
## 4:	months	0.074874299	0.051085111	0.125842183
## 5:	largest	0.054948428	0.003854733	0.001712915
## 6:	fireplace	0.043052019	0.012229033	0.019984013
## 7:	fair	0.037429677	0.017870090	0.022838872
## 8:	qabove_avg	0.032105418	0.005319850	0.005252941
## 9:	ecf2R214	0.025701137	0.017767064	0.010620075
## 10:	ln_garagespaces	0.023976436	0.016556186	0.046933881
## 11:	two_plus_baths	0.018775971	0.002356580	0.001827110
## 12:	poor	0.018412480	0.017654504	0.011990408
## 13:	powder_room	0.012633399	0.002777039	0.006280690
## 14:	good	0.012023311	0.015596526	0.007194245
## 15:	ecf3R310	0.011157488	0.018904224	0.005595524
## 16:	large	0.009811609	0.002661647	0.008221994
## 17:	smallest	0.009410634	0.002340487	0.002512276
## 18:	central_air	0.008430529	0.009258920	0.012332991
## 19:	small	0.006687884	0.002675617	0.006280690
## 20:	single_family1story	0.006141464	0.004431171	0.014274295
##	Feature	Gain	Cover	Frequency

```
# art <- save_avm_bundle(fit)
# message("\nSaved artifacts:")
# print(art)
```

Model IV

```
formula_ln_tasp <- ln_tasp_re ~
  ln_landratio + ln_base_bldgsize_ratio + arterial +
  single_family1story + half_duplex1story + half_duplex2story +
  smallest + small + large + largest + two_plus_baths + powder_room +
  good + fair + poor + very_poor_unsound + qabove_avg + qbelow_avg +
  sresb_yearbuilt + crawl + slab + ln_garagespaces +
  fireplace + hot_water + elec_baseboard + wall + central_air + months + ecf

message("Training LightGBM AVM with formula-driven features...")
mod_tasp <- train_lgbm_avm(df, formula = formula_ln_tasp)

message("\nHoldout metrics:")
print(mod_tasp$metrics)
```

```
## $rmse
## [1] 0.4233119
##
## $mae
## [1] 0.3504959
##
## $r2
## [1] 0.4364293
```

```
message("\nTop 20 features by Gain:")
print(head(mod_tasp$importance[order(-Gain)], 20))
```

##	Feature	Gain	Cover	Frequency
##	<char>	<num>	<num>	<num>
## 1:	ln_base_bldgsize_ratio	0.473381185	0.149246878	0.227148331
## 2:	ln_landratio	0.092628454	0.126873152	0.156540777
## 3:	sresb_yearbuilt	0.084092706	0.128327061	0.180213465
## 4:	months	0.050311526	0.054563438	0.141488779
## 5:	qabove_avg	0.047551645	0.005112184	0.005062945
## 6:	fair	0.034441698	0.022113076	0.023125342
## 7:	fireplace	0.030765558	0.013436815	0.020114943
## 8:	ecf2R214	0.019795514	0.018505898	0.011220580
## 9:	largest	0.013168388	0.002398122	0.001094691
## 10:	central_air	0.011373338	0.013132674	0.016830870
## 11:	poor	0.009429405	0.020551610	0.006704981
## 12:	ecf3R319	0.009277036	0.019120518	0.007115490
## 13:	ecf1R126	0.008303503	0.018669719	0.004652436
## 14:	good	0.006875586	0.021287243	0.005747126
## 15:	qbelow_avg	0.006699899	0.010817437	0.016967707
## 16:	ecf7R702	0.006141114	0.015253080	0.004652436
## 17:	ln_garagespaces	0.005913255	0.006981398	0.016830870
## 18:	ecf6R620	0.005578315	0.018686588	0.004652436
## 19:	ecf4R411	0.005263686	0.019105501	0.005336617
## 20:	large	0.005094623	0.002940193	0.004515599
##	Feature	Gain	Cover	Frequency

```
# art <- save_avm_bundle(fit)
# message("\nSaved artifacts:")
# print(art)
```

Model V

```
formula_alt <- ln_tasp_re ~
  ln_landratio + lnsbldsint + arterial + single_family1story + half_duplex1story +
  half_duplex2story + smallest + small + large + largest + two_plus_baths + powder_room +
  good + fair + poor + very_poor_unsound + qabove_avg + qbelow_avg +
  sresb_yearbuilt + crawl + slab + garage + fireplace +
  hot_water + elec_baseboard + wall + central_air + months + ecf

message("Training LightGBM AVM with formula-driven features...")
mod_alt <- train_lgbm_avm(df, formula = formula_alt)

message("\nHoldout metrics:")
print(mod_alt$metrics)
```

```
## $rmse
## [1] 0.4220025
##
## $mae
## [1] 0.3506246
##
## $r2
## [1] 0.4448414
```

```
message("\nTop 20 features by Gain:")
print(head(mod_alt$importance[order(-Gain)], 20))
```

	Feature	Gain	Cover	Frequency
	<char>	<num>	<num>	<num>
## 1:	lnsbldsint	0.419052351	0.135404294	0.218068536
## 2:	ln_landratio	0.087644019	0.115349878	0.161103694
## 3:	sresb_yearbuilt	0.080505103	0.122306801	0.178163477
## 4:	months	0.053106724	0.061906880	0.153983089
## 5:	qabove_avg	0.044852277	0.009954948	0.006823913
## 6:	largest	0.043285553	0.004461680	0.001483459
## 7:	fireplace	0.042522011	0.016062774	0.018839935
## 8:	fair	0.039037398	0.024650058	0.023735351
## 9:	ecf2R214	0.023642853	0.020050577	0.011274292
## 10:	ecf3R319	0.012720354	0.017110574	0.007862335
## 11:	large	0.012377941	0.004697712	0.006675567
## 12:	poor	0.011124682	0.020841119	0.007268951
## 13:	central_air	0.010650107	0.013632592	0.017949859
## 14:	ecf1R126	0.009717085	0.020653616	0.005488800
## 15:	qbelow_avg	0.007246021	0.011664768	0.016614746
## 16:	good	0.007113837	0.021169943	0.005785492
## 17:	ecf4R411	0.006391506	0.021417921	0.005933838
## 18:	ecf1R104	0.005879022	0.011570803	0.005488800
## 19:	ecf7R702	0.005733963	0.015717952	0.005043762
## 20:	ecf4R401	0.005535441	0.019767935	0.005192108
##	Feature	Gain	Cover	Frequency

```
# art <- save_avm_bundle(fit)
# message("\nSaved artifacts:")
# print(art)
```

Model VI

```
formula_alt2 <- ln_tasp_re ~
  ln_landratio + lnsbldsfmt + arterial + single_family1story + half_duplex1story +
  half_duplex2story + smallest + small + large + largest + two_plus_baths + powder_room +
  good + fair + poor + very_poor_unsound + qabove_avg + qbelow_avg +
  sresb_yearbuilt + crawl + slab + garage + fireplace +
  hot_water + elec_baseboard + wall + central_air + ecf

message("Training LightGBM AVM with formula-driven features...")
mod_alt2 <- train_lgbm_avm(df, formula = formula_alt2)

message("\nHoldout metrics:")
print(mod_alt2$metrics)
```

```
## $rmse
## [1] 0.4307583
##
## $mae
## [1] 0.3580945
##
## $r2
## [1] 0.4247561
```

```
message("\nTop 20 features by Gain:")
print(head(mod_alt2$importance[order(-Gain)], 20))
```

##	Feature	Gain	Cover	Frequency
##	<char>	<num>	<num>	<num>
## 1:	lnsbldsfmt	0.512959088	0.171469147	0.2851605759
## 2:	ln_landratio	0.086785599	0.116011254	0.1795865633
## 3:	sresb_yearbuilt	0.080388586	0.123323303	0.1954595792
## 4:	qabove_avg	0.056475787	0.008047684	0.0081210779
## 5:	fair	0.041217298	0.027479237	0.0324843116
## 6:	fireplace	0.029890101	0.019959869	0.0236249539
## 7:	ecf2R214	0.023302967	0.023578887	0.0153193060
## 8:	largest	0.012877540	0.001067263	0.0009228498
## 9:	poor	0.010938701	0.023728958	0.0077519380
## 10:	central_air	0.010818662	0.017782816	0.0265780731
## 11:	ecf1R126	0.009751554	0.024503440	0.0060908084
## 12:	ecf3R319	0.008824715	0.017758144	0.0088593577
## 13:	good	0.008409729	0.023813603	0.0060908084
## 14:	ecf4R411	0.007261679	0.024049001	0.0064599483
## 15:	qbelow_avg	0.007176334	0.011699687	0.0164267257
## 16:	ecf2R203	0.006662613	0.015888185	0.0062753784
## 17:	ecf1R104	0.006266231	0.012154535	0.0066445183
## 18:	ecf6R620	0.005768763	0.018733130	0.0046142488
## 19:	ecf7R702	0.005414930	0.014911427	0.0049833887
## 20:	ecf4R401	0.005261608	0.019194656	0.0051679587
##	Feature	Gain	Cover	Frequency

```
# art <- save_avm_bundle(fit)
# message("\nSaved artifacts:")
# print(art)
```

```
# -----  
# Example prediction on a single sale/parcel  
# -----  
# Suppose your df has sale_id or parcelno:  
# new_row <- newdata_from_id(df, "sale_id", 123456)  
# yhat <- predict_auv(art$bundle_file, new_row)  
# print(yhat)
```

```
diffdf(dfa, dfc)
```